

Laboratorio 1

Indice

1	Risorse	5
2	Concetti di base	6
2.1	Compilatore	6
2.2	Interprete	6
2.3	Transpilatore	6
3	Architettura di un computer	7
3.1	Ciclo fetch-execute	7
4	Rappresentazione binaria	8
4.1	Da base 10 a base 2	8
4.2	Perché si usa	8
4.3	Overflow	9
4.4	Rappresentazione con modulo e segno	9
4.5	Complemento a 1	9
4.6	Complemento a 2	9
4.7	Numeri reali in base 2 (IEEE 754)	10
4.7.1	Campi dello standard IEEE 754	10
4.7.2	Valori speciali dell'esponente	11
4.7.3	Intervalli di rappresentazione	12
4.8	Rappresentazione dei caratteri testuali in binario: ASCII e Unicode	12
4.9	Immagini raster (bitmap)	13
4.10	Campionamento dei suoni in binario	14
4.10.1	Campionamento	14
4.10.2	Quantizzazione	14
4.10.3	Esempio pratico	15
4.10.4	Rappresentazione grafica	15
4.10.5	Esempio binario finale	15
4.11	Operatori binari logici e di scorrimento	16
4.11.1	Operatori logici	16
4.11.2	Operatori di scorrimento (Bit shift)	17

5	JavaScript	18
5.1	Tipi di dati	18
5.2	Numeri in JavaScript	18
5.3	Costrutti	19
5.4	Insiemi in JavaScript	19
5.5	Semantica per riferimento	19
6	Domande da esame	20
6.1	Qual'è la differenza tra var, let e const?	20
6.2	Qual è la differenza tra __proto__ e .prototype?	20
6.3	Come funzionano gli access modifier e quali sono?	21
6.4	A cosa servono getter e setter?	22
6.5	Cosa significa che una classe è una sottoclasse o che una classe estende un'altra classe?	22
6.6	Qual'è la differenza tra semantica per valore e semantica per riferimento?	22
6.7	Cos'è l'hoisting?	23
6.8	Cosa sono le interfacce e a cosa servono?	23
6.9	Cosa sono i generics e a cosa servono?	24
6.10	Qual'è la differenza tra tipizzazione in JavaScript e TypeScript?	24
6.11	Se assegno una stringa a una variabile in JavaScript, posso poi assegnarle un numero?	24
6.12	Come si chiama il fatto che in TypeScript una variabile non possa cambiare tipo?	25
6.13	Differenza tra any e unknown?	25
6.14	Cos'è un modulo e a cosa serve?	25
6.15	Quale parola chiave si usa per esportare una classe?	25
6.16	Come si fa a fare in modo che una variabile possa assumere solo determinati valori specifici?	25
6.17	Qual'è la differenza tra interface extends interface e class ex- tends class?	26
6.18	Se ho un oggetto di una classe Figlio che estende una classe che implementa un'interfaccia, che cosa si trova nel prototype?	26
6.19	Cos'è una Map?	26
6.20	Dove vengono salvati metodi e attributi ereditati di una classe?	26
6.21	Come modificare metodi e aggiungerne tramite prototype?	27
6.22	Qual è la differenza tra funzione normale e arrow function?	27
6.23	Come funziona il this in JavaScript?	28

6.24	Qual è la differenza tra shallow copy e deep copy?	29
6.25	Qual è la differenza tra == e ===?	29
6.26	Cos'è il type coercion in JavaScript?	30
6.27	Qual è la differenza tra null e undefined?	30
6.28	Cosa sono i tipi union e intersection in TypeScript?	31
6.29	Cos'è il type inference in TypeScript?	31
6.30	Qual è la differenza tra type e interface in TypeScript?	32
6.31	Cos'è una classe astratta (abstract class)?	32
6.32	Cosa succede quando TypeScript viene compilato in JavaScript?	32
6.33	Cos'è il runtime e cos'è il compile-time?	33
6.34	Differenza tra Map e Object	33
6.35	Cos'è una Promise e come funziona l'asincronia in JavaScript?	34
6.36	Cos'è l'event loop?	34
6.37	Differenza tra stack, heap e garbage collector	35
6.38	Cosa sono i decorator in TypeScript?	35

1 Risorse

- Libro di testo: eloquentjavascript.net
- IDE online TypeScript: typescriptlang.org/play

2 Concetti di base

2.1 Compilatore

- Crea un file eseguibile a partire dal codice sorgente.
- **Scanner**: analisi lessicale.
- **Parser**: analisi sintattica.
- **Type-checker**: analisi semantica.
- **Linker**: collega librerie e file.

2.2 Interprete

- Esegue il codice direttamente, istruzione per istruzione.
- Usa il ciclo **fetch-execute**.

2.3 Transpilatore

- Converte codice da un linguaggio a un altro.
- Esempio: TypeScript viene convertito in JavaScript.

3 Architettura di un computer

- **CPU** = processore:
 - **ALU**: Arithmetic Logic Unit
 - **CU**: Control Unit
- **System Bus**: canale di comunicazione
- **I/O**: periferiche
- **Memoria**: RAM, SSD, registri

3.1 Ciclo fetch-execute

1. Recupera istruzione dalla RAM
2. Decodifica
3. Esecuzione
4. Ripeti

4 Rappresentazione binaria

Come la rappresentazione posizionale in base 10 permette questo:

$$\mathbf{7456} = (7 \cdot 1000) + (4 \cdot 100) + (5 \cdot 10) + (6 \cdot 1) = (7 \cdot 10^3) + (4 \cdot 10^2) + (5 \cdot 10^1) + (6 \cdot 10^0)$$

$$\text{Cifre} = \{0, 1, \dots, \text{Base} - 1\}$$

$$\text{Base} = 10$$

La rappresentazione in base 2 (**binaria**) permette questo:

$$\mathbf{1010} = (1 \cdot 2^3) + (1 \cdot 2^1) = 10 \text{ (in base 10)}$$

$$\text{Cifre} = \{0, 1\} = \{0, 2 - 1\}$$

4.1 Da base 10 a base 2

Divisione per 2	Quoziente	Resto (bit)
$154 \div 2$	77	0
$77 \div 2$	38	1
$38 \div 2$	19	0
$19 \div 2$	9	1
$9 \div 2$	4	1
$4 \div 2$	2	0
$2 \div 2$	1	0
$1 \div 2$	0	1
Risultato: $154_{10} = 10011010_2$		

Per appunto trasformare un numero da base 10 a base 2 occorre fare il modulo più volte su di esso fino ad arrivare a 0.

4.2 Perché si usa

I computer lavorano più efficientemente in base 2, dato che utilizzano soltanto due valori **ON/OFF**, indicati da variazione di tensione 0v e +5v.

4.3 Overflow

Se abbiamo n bit a disposizione per rappresentare dei numeri ed effettuiamo un'operazione tra due numeri il cui risultato non può essere rappresentato in n bit si ha un **overflow**.

L'overflow è causa di errore di calcolo, in quanto, se non curato, esclude le cifre che non riescono a essere rappresentate.

b_0 = bit meno significativo (**Least Significant Bit**, LSb)

b_{n-1} = bit più significativo (**Most Significant Bit**, MSb)

4.4 Rappresentazione con modulo e segno

Per rappresentare gli interi in base 2 bisogna riuscire a rappresentare anche il segno.

Per farlo, si riserva un bit per il $+$ e il $-$, spesso il **MSb**.

Con n bit, possiamo rappresentare gli interi $\in [-2^{(n-1)}, 2^{n-1} - 1]$

4.5 Complemento a 1

Quando si esegue una somma si dovrebbero verificare i segni dei due numeri e, se risultano diversi, si dovrebbe eseguire una sottrazione. Ciò vuol dire che prima di ogni operazione il calcolatore deve fare una verifica in più.

Per risolvere, si usa il **complemento a 1**, ovvero il NOT su ogni bit.

Adesso la somma tra numeri negativi e positivi funziona, risultando però spesso con overflow che deve essere riaggiunto al numero.

Il problema del complemento a 1 è che ha due rappresentazioni per lo 0:

$$0^+ = 00000000$$

$$0^- = 11111111$$

4.6 Complemento a 2

Il **complemento a 2** risolve il problema della doppia rappresentazione dello 0 del complemento a 1.

Vantaggi complemento a 2:

- Un solo 0 (0000 0000, non ho più $0^+, 0^-$), se faccio il complemento a 2 di 0 è sempre 0000 0000.
- L'operazione somma non ha bisogno di controllare il segno.
- Scarta l'overflow.

La sottrazione $a - b$ è la somma $a + \bar{\bar{b}}$.

Per trovare $\bar{\bar{b}}$ si deve prima fare il complemento a 1 di b e poi aggiungere 1.

4.7 Numeri reali in base 2 (IEEE 754)

IEEE 754 è lo standard internazionale per la rappresentazione dei numeri in virgola mobile (**floating-point**), usato praticamente da tutti i computer e linguaggi di programmazione.

4.7.1 Campi dello standard IEEE 754

Campo	Significato	Bit (32 bit)	Bit (64 bit)
Sign (S)	segno del numero	1	1
Exponent (E)	esponente in base 2	8	11
Mantissa / Fraction (M)	parte frazionaria del numero normalizzato	23	52

Tabella 1: Struttura dei campi per i formati IEEE 754 a 32 e 64 bit

Bit di segno (Sign bit) Il bit di segno indica il segno del numero rappresentato:

$$S = 0 \rightarrow S \geq 0$$

$$S = 1 \rightarrow S < 0$$

Esponente (Exponent) L'esponente determina la potenza di 2 per cui va moltiplicata la mantissa, cioè sposta la virgola binaria a destra o a sinistra. Per evitare di memorizzare esponenti negativi, lo standard IEEE 754 utilizza un valore chiamato *bias* (o offset). In pratica, l'esponente memorizzato in memoria non è quello reale, ma:

$$E_{reale} = E_{memorizzato} - bias$$

Il valore del bias dipende dal formato:

Tipo	Bit Esponente	Bias
Float (32 bit)	8	127
Double (64 bit)	11	1023

Per esempio, nel formato float (32 bit):

$$E_{\text{memorizzato}} = 10000001_2 = 129_{10}$$

$$E_{\text{reale}} = 129 - 127 = 2$$

Quindi l'esponente effettivo è 2, che equivale a moltiplicare la mantissa per $2^2 = 4$.

Mantissa (Fraction) La mantissa rappresenta la parte frazionaria del numero normalizzato. Nei numeri normalizzati, si assume sempre la presenza di un 1 implicito prima del punto binario, quindi il valore effettivo è:

$$1.\text{mantissa bits}$$

Esempio:

$$\text{Mantissa} = 010000000000000000000000$$

$$\Rightarrow 1.01_2 = 1 + 0.25 = 1.25$$

In altre parole, la mantissa determina la precisione del numero, mentre l'esponente ne controlla la scala, cioè quanto grande o piccolo è il valore.

4.7.2 Valori speciali dell'esponente

Esponente	Mantissa	Significato
00000000	000...0	0 (positivo o negativo)
00000000	$\neq 0$	Numero denormalizzato (molto vicino a 0)
11111111	000...0	∞ (positivo o negativo)
11111111	$\neq 0$	NaN (Not a Number)

Tabella 2: Valori speciali dell'esponente nello standard IEEE 754

4.7.3 Intervalli di rappresentazione

- **DP (Double Precision)**: 64 bit totali, esponente 11 bit, mantissa 52 bit, bias 1023.
- **SPS (Single Precision)**: 32 bit totali, esponente 8 bit, mantissa 23 bit, bias 127.
- **QP (Quad Precision)**: 128 bit totali, esponente 15 bit, mantissa 112 bit, bias 16383.

L'intervallo dei numeri rappresentabili dipende dal valore massimo e minimo dell'esponente:

Protocollo	Valore minimo positivo	Valore massimo
SPS (32 bit)	$\approx 1.175 \cdot 10^{-38}$	$\approx 3.402 \cdot 10^{38}$
DP (64 bit)	$\approx 2.225 \cdot 10^{-308}$	$\approx 1.798 \cdot 10^{308}$
QP (128 bit)	$\approx 3.362 \cdot 10^{-4932}$	$\approx 1.189 \cdot 10^{4932}$

4.8 Rappresentazione dei caratteri testuali in binario: ASCII e Unicode

I caratteri testuali nei computer sono rappresentati tramite codifiche standard, assegnando a ciascun simbolo un numero binario unico.

ASCII (American Standard Code for Information Interchange)

- Codifica a 7 bit (128 caratteri).
- Include lettere maiuscole e minuscole, numeri, simboli di punteggiatura e caratteri di controllo.

Unicode UTF-8

- Codifica variabile (1-4 byte), compatibile con ASCII.
- Permette di rappresentare tutti i caratteri del mondo (simboli, lettere accentate, ideogrammi, emoji).

4.9 Immagini raster (bitmap)

Le immagini raster, o bitmap, sono composte da **pixel**, ovvero piccoli punti che contengono informazioni sul colore e sulla luminosità. Ogni pixel è rappresentato in binario, e il numero di bit determina la profondità di colore e la precisione dell'immagine.

Bianco e nero

- Ogni pixel occupa **1 bit**.
- Valori possibili:
 - 0 = bianco
 - 1 = nero
- È la rappresentazione più semplice, usata in fax, testi scansionati o icone.

Scala di grigio

- Ogni pixel occupa **8 bit** (1 byte).
- Valori possibili: da 0 a 255, dove:
 - 0 = nero
 - 255 = bianco
 - valori intermedi = sfumature di grigio
- Utilizzata in fotografia in bianco e nero e immagini medicali.

Colori RGB

- Ogni pixel occupa **24 bit** (3 byte), uno per ciascun canale dei colori primari:
 - Rosso (R)
 - Verde (G)
 - Blu (B)

- Ogni canale varia da 0 a 255, quindi i valori RGB combinati definiscono il colore finale del pixel.

Esempio:

- $\text{RGB}(255,0,0)$ = rosso pieno
- $\text{RGB}(0,255,0)$ = verde pieno
- $\text{RGB}(0,0,255)$ = blu pieno
- $\text{RGB}(255,255,255)$ = bianco
- $\text{RGB}(0,0,0)$ = nero

4.10 Campionamento dei suoni in binario

Il suono è un segnale analogico continuo, cioè varia nel tempo in modo costante. Per poter essere memorizzato ed elaborato da un computer, deve essere convertito in una forma digitale, cioè in una sequenza di numeri binari. Questo processo si chiama **digitalizzazione del suono** e si compone di due fasi principali: **campionamento** e **quantizzazione**.

4.10.1 Campionamento

Il **campionamento** consiste nel misurare a intervalli regolari di tempo l'ampiezza del segnale sonoro. Ogni misurazione è chiamata *campione*. La frequenza con cui vengono presi i campioni è detta **frequenza di campionamento** e si misura in Hertz (Hz).

Ad esempio:

- 8 kHz $\rightarrow$ qualità telefonica
- 44.1 kHz $\rightarrow$ qualità CD audio
- 48 kHz o superiore $\rightarrow$ qualità professionale

4.10.2 Quantizzazione

Dopo il campionamento, ogni valore analogico viene trasformato in un numero digitale. Il numero di **bit** usati per rappresentare ciascun campione determina la **profondità di bit** (bit depth). Con n bit, si possono rappresentare 2^n livelli di ampiezza.

$$\text{Numero di livelli} = 2^n$$

Ad esempio, con 16 bit si hanno $2^{16} = 65536$ livelli.

4.10.3 Esempio pratico

Supponiamo che un segnale sonoro vari tra -1 e $+1$ volt e che venga quantizzato con 3 bit. Ogni livello di quantizzazione avrà un valore binario come in tabella:

Ampiezza (V)	Codice binario (3 bit)
+1.00	111
+0.75	110
+0.50	101
+0.25	100
-0.25	011
-0.50	010
-0.75	001
-1.00	000

Tabella 3: Esempio di quantizzazione di un segnale sonoro con 3 bit

Ogni valore di ampiezza viene così trasformato in una sequenza binaria, che può essere memorizzata in un file audio (ad esempio, WAV o PCM).

4.10.4 Rappresentazione grafica

Il seguente schema mostra il processo di digitalizzazione del suono:

Segnale analogico $\xrightarrow[\text{campionamento}]{\text{a intervalli regolari}}$ campioni discreti $\xrightarrow[\text{quantizzazione}]{\text{in bit}}$ valori binari

4.10.5 Esempio binario finale

Un breve estratto di campioni potrebbe risultare così:

$$\text{Ampiezza} \rightarrow \{-0.75, -0.25, +0.50, +1.00\}$$

$$\text{Codifica binaria} \rightarrow \{001, 011, 101, 111\}$$

Questa sequenza di bit rappresenta il suono originale in formato digitale.

4.11 Operatori binari logici e di scorrimento

In informatica, i dati binari possono essere manipolati tramite **operatori binari**, che agiscono direttamente sui singoli bit. Questi operatori si dividono principalmente in due categorie: **operatori logici** e **operatori di scorrimento**.

4.11.1 Operatori logici

Gli **operatori logici bit a bit** confrontano i bit di due operandi e restituiscono un nuovo valore binario come risultato. Di seguito sono mostrati i principali operatori logici binari:

Operatore	Nome	Simbolo
AND	Conjunctione logica	$\&$
OR	Disgiunzione logica	—
XOR	Disgiunzione esclusiva	$\oplus$
NOT	Negazione logica	$\sim$

Tabella 4: Operatori logici binari

Ecco le **tabelle di verità** per ciascun operatore binario:

A	B	$A \& B$	$A \text{ — } B$	$A \oplus B$
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Tabella 5: Tabelle di verità per AND, OR e XOR

Esempio pratico con numeri binari a 4 bit:

$$\begin{array}{rcl}
 A & = & 1010 \\
 B & = & 1100 \\
 \hline
 A \& B & = & 1000 \\
 A | B & = & 1110 \\
 A \oplus B & = & 0110 \\
 \sim A & = & 0101
 \end{array}$$

4.11.2 Operatori di scorrimento (Bit shift)

Gli **operatori di scorrimento** spostano tutti i bit di un numero verso sinistra o verso destra. Ogni spostamento equivale a una moltiplicazione o divisione per 2 (nelle rappresentazioni senza segno).

Operatore	Simbolo	Descrizione
Shift a sinistra	<<	Sposta tutti i bit a sinistra, inserendo zeri a destra
Shift a destra	>>	Sposta tutti i bit a destra, eliminando i bit a destra
Shift a destra con inserimento di 0	>>>	Come lo shift a destra, ma inserisce 0 a sinistra (non mantiene il segno)

Tabella 6: Operatori binari di scorrimento

Esempio:

$$A = 00101100$$

$$A \ll 2 = 10110000 \quad (\text{spostato a sinistra di 2, moltiplicato per 4})$$

$$A \gg 2 = 00001011 \quad (\text{spostato a destra di 2, diviso per 4})$$

$$A \ggg 2 = 00001011 \quad (\text{come sopra, ma con zeri inseriti a sinistra})$$

5 JavaScript

- Nato nel 1995 per il browser Netscape.
- Inizialmente client-side, ora anche server-side.
- **Node.js**: ambiente per esecuzione di codice JavaScript.
- **ECMAScript 2025**: ultima versione (settembre 2025).
- Linguaggio interpretato, flessibile e poco tipizzato (*weakly typed*).
- Difficile trovare errori, ma adatto a soluzioni creative.

5.1 Tipi di dati

- Numeri (Insiemi numerici, floating point...)
- BigInt
- Booleani (T,F)
- Stringhe ("Ciao", "X vale -X")
- `undefined`
- `null`
- Oggetti
- Array
- Funzioni

5.2 Numeri in JavaScript

- Interi e reali
- Valori speciali: NaN, Infinity, -Infinity

5.3 Costrutti

- If-Else
 - Si può svolgere anche così: "case ? trueCond : falseCond"
- Switch
- For, ForEach (sempre un "for")
- While, Do-While

5.4 Insiemi in JavaScript

Collezione di elementi distinti e non ordinati.

$$\{mela, pera, banana\}, \{1, 2, 3, 4\}$$

Si rappresentano tramite l'utilizzo di oggetti.

$$let A = \{\}$$

5.5 Semantica per riferimento

In JavaScript, la **semantica per riferimento** permette la condivisione di oggetti in memoria.

$$vara = [1, 2, 3]$$
$$varc = a$$

In questo caso, se a viene modificato, anche c viene modificato, perchè hanno lo stesso riferimento in memoria.

Un esempio con un grafo, per insistere che più nodi sono collegati al solito nodo, si usano i riferimenti e non i valori singoli.

$$n1 = 1, n2 = 2, n3 = 3, n4 = 4$$
$$nodo1 = [n2, n4]$$
$$nodo2 = [n2, n4]$$
$$nodo3 = [n1, n4]$$
$$nodo4 = [n1]$$
$$G = [n1, n2, n3, n4]$$

6 Domande da esame

6.1 Qual'è la differenza tra var, let e const?

La prima differenza riguarda lo scoping:

- **var:** visibile in tutta la funzione anche se dichiarato in un blocco più interno. (ma non esce mai fuori da una funzione). Se dichiarato in un costrutto non di funzione allora è visibile all'esterno di tale blocco.
- **let e const:** scoping a blocchi, non visibili all'esterno del blocco in cui sono stati dichiarati.

Continuando, si ha differenza riguardo l'hoisting:

- **var:** la variabile viene hoistata e può essere utilizzata prima della dichiarazione, ma non essendo ancora stata sottoposta ad assegnazioni il valore è "undefined"
- **let e const:** la variabile viene hoistata ma non può essere usata finché non si effettua un'assegnazione. Lo stato in cui tale variabile esiste ma non può essere usata si chiama **Temporal Dead Zone (TDZ)**.

Quindi si può dire che:

- **var:** function scope
- **let e const:** block scope

Inoltre, const non può essere riassegnata al contrario delle altre due.

Infine, solo var permette la ridichiarazione, ovvero dichiarare una variabile con lo stesso nome e la stessa keyword

6.2 Qual è la differenza tra __proto__ e .prototype?

In JavaScript esiste un sistema di ereditarietà prototipale basato sui prototipi.

La proprietà **prototype** appartiene alle funzioni costruttrici e alle classi, e rappresenta l'oggetto che verrà utilizzato come prototipo delle istanze create tramite la keyword **new**.

La proprietà `__proto__`, invece, appartiene alle singole istanze e rappresenta il riferimento al prototipo effettivo dell'oggetto.

Le due proprietà sono strettamente collegate: quando viene creata un'istanza tramite una classe o una funzione costruttrice, il valore di `__proto__` dell'istanza coincide con il valore di `prototype` della classe o funzione utilizzata per crearla.

In altre parole:

```
istanza.__proto__ === Classe.prototype
```

restituisce `true`.

Per esempio:

```
class User {}  
const u = new User();  
console.log(u.__proto__ === User.prototype);
```

In questo caso il risultato sarà `true`.

Il meccanismo dei prototipi permette di condividere metodi e proprietà tra tutte le istanze senza duplicare il codice in memoria.

È importante notare che `__proto__` è considerata una proprietà legacy; nelle applicazioni moderne si preferisce utilizzare metodi standard come:

```
Object.getPrototypeOf(obj)
```

e

```
Object.setPrototypeOf(obj, proto)
```

6.3 Come funzionano gli access modifier e quali sono?

I modificatori d'accesso permettono di modificare la visibilità di attributi e metodi di una classe e sono:

- **Private:** rende un attributo accessibile solo all'interno della classe.
- **Public:** rende un attributo accessibile anche al di fuori della classe.
- **Protected:** rende un attributo accessibile solo all'interno della classe e delle sottoclassi.

Questi possono essere utilizzati con le loro rispettive keyword da inserire prima dell'identificativo di un attributo o un metodo.

6.4 A cosa servono getter e setter?

I metodi getter servono per restituire i valori degli attributi di un istanza, i metodi setter invece servono per la modifica.

L'esistenza di questi metodi garantisce l'information hiding e l'incapsulamento.

Si crea un layer di controllo che permette di rendere più moderato il processo di restituzione valori e modifica.

Per esempio, se vogliamo aggiornare un campo con un nuovo valore bisogna controllare che tale valore sia valido.

Come ulteriore esempio, potremmo rifiutare di restituire un valore se si avvera una condizione, se per esempio non si possiedono le credenziali giuste.

6.5 Cosa significa che una classe è una sottoclasse o che una classe estende un'altra classe?

In JavaScript dire che una classe è una sottoclasse o che estende un'altra classe significa che la prima classe eredita proprietà e metodi della seconda.

La classe da cui si eredita si chiama superclasse (o classe padre), mentre quella che eredita si chiama sottoclasse (o classe figlia).

Il costruttore della sottoclasse richiama il costruttore della superclasse per evitare di riscrivere le stesse cose tramite il metodo `super(...args)`.

In pratica la parola chiave usata "extends" allunga la prototype chain collegando i prototype della superclasse e sottoclasse, si crea un riferimento nel prototype della sottoclasse che riporta a quello della superclasse.

6.6 Qual'è la differenza tra semantica per valore e semantica per riferimento?

In JavaScript tutti i passaggi avvengono per valore, ma per gli oggetti il valore copiato è un riferimento.

Quando si assegna una variabile ad un'altra le due variabili rimangono indipendenti e modificare la prima non significa modificare la seconda.

Negli oggetti invece si crea un riferimento al secondo oggetto e se dovessimo modificare il campo che è stato assegnato ad un oggetto si andrebbe a modificare anche l'oggetto stesso.

6.7 Cos'è l'hoisting?

L'hoisting consiste nel comportamento per cui le dichiarazioni di variabili e funzioni vengono "spostate" logicamente all'inizio del loro scope durante la fase di compilazione.

Ogni dichiarazione è spostata all'inizio dello scope.

Basta ricordare come le funzioni possono essere chiamate con una chiamata scritta "prima" della sua definizione.

Facciamo un esempio di come l'hoisting "sposta" le dichiarazioni all'inizio dello scope:

```
console.log(x)
var x = 10
```

Questo al momento di compilazione / interpretazione diventa:

```
var x
console.log(x)
x = 10
```

Nel caso delle funzioni però non viene spostata solo la dichiarazione, ma TUTTA la definizione:

```
greet()
function greet() {
  console.log("hi")
}
```

Diventa:

```
function greet() {
  console.log("hi")
}
greet()
```

6.8 Cosa sono le interfacce e a cosa servono?

Le interfacce sono **tipi astratti** che definiscono una struttura / logica che delle istanze devono seguire.

In JavaScript non esistono le interfacce e al tempo di run tramite TS spariscono.

le interfacce specificano le caratteristiche di tale struttura senza però implementarla, senza attribuire valori o implementare metodi.

6.9 Cosa sono i generics e a cosa servono?

I Generics sono una funzionalità di TypeScript che permette di scrivere codice riutilizzabile e tipizzato, senza dover specificare in anticipo il tipo dei dati con cui lavorerà. In pratica, consentono di creare funzioni, classi e interfacce che possono operare su tipi diversi mantenendo comunque il controllo del tipo a compile-time.

Il vantaggio principale è evitare duplicazioni di codice e aumentare la sicurezza del programma. Ad esempio, invece di scrivere una funzione per le stringhe e una per i numeri, si scrive una sola funzione generica che funziona con qualsiasi tipo.

6.10 Qual'è la differenza tra tipizzazione in JavaScript e TypeScript?

Le differenze principali sono queste:

- **JavaScript:** tipizzazione dinamica, weakly typed, il tipo delle variabili è deciso durante il run-time e una variabile può assumere diversi tipi di valore nel corso dell'esecuzione.
- **TypeScript:** tipizzazione statica, strongly typed, il tipo delle variabili è deciso al compile-time e c'è un type-checking precoce che segnala eventuali errori.

In pratica TypeScript offre maggiore sicurezza, JavaScript garantisce più flessibilità.

6.11 Se assegno una stringa a una variabile in JavaScript, posso poi assegnarle un numero?

Sì, appunto per il discorso della tipizzazione dinamica.

6.12 Come si chiama il fatto che in TypeScript una variabile non possa cambiare tipo?

Tipizzazione statica.

6.13 Differenza tra `any` e `unknown`?

In TypeScript questi due tipi indicano che l'oggetto non ha un tipo specifico ma ci sono particolari differenze:

- **Any:** accetta un qualsiasi tipo e quindi disabilita il type checking
- **Unknown:** indica che non si conosce ancora quale tipo potrebbe assumere tale oggetto e servono quindi dei controlli sul tipo prima di eseguire una certa operazione.

6.14 Cos'è un modulo e a cosa serve?

Un file JS o TS può esportare il suo contenuto, come funzioni, costanti e altro, per renderlo disponibile ad altri file. Ogni volta che un file esporta qualcosa diventa un modulo.

I moduli servono per rendere riutilizzabili parti di codice e organizzare il progetto in molte parti indipendenti. (Anche i file che importano solo moduli)

6.15 Quale parola chiave si usa per esportare una classe?

Export, import serve invece per importare.

6.16 Come si fa a fare in modo che una variabile possa assumere solo determinati valori specifici?

In tal caso si usa un **literal type**, ovvero una cosa del genere (TypeScript):

```
let temp = "hi" | 21 | true
```

Altrimenti si possono usare gli enum.

6.17 Qual'è la differenza tra `interface extends interface` e `class extends class`?

`Interface extends interface` serve solo per realizzare dei tipi, viene eliminata in fase di compilazione e non usa la prototype chain.

Invece `class extends class` crea una vera e propria ereditarietà ed usa la prototype chain.

6.18 Se ho un oggetto di una classe Figlio che estende una classe che implementa un'interfaccia, che cosa si trova nel prototype?

6.19 Cos'è una Map?

Map in TypeScript e JavaScript è una struttura dati / collezione che memorizza associazioni chiave-valore.

Molto simile ad un oggetto, ma è flessibile e moderna.

Una differenza sta nei tipi accettabili delle chiavi:

- **Oggetto qualsiasi:** le chiavi possono essere solo di tipo string o symbol.
- **Map:** le chiavi possono essere di qualsiasi tipo.

In TypeScript si può specificare il tipo delle chiavi e dei valori di una Map tramite i suoi generics.

I metodi principali sono set, get, has, delete.

6.20 Dove vengono salvati metodi e attributi ereditati di una classe?

Gli attributi vengono salvati direttamente nell'istanza, nel prototype vengono salvati solo i metodi, sia i propri che quelli ereditati.

I metodi si possono accedere appunto da i due campi dei prototipi visti in precedenza e gli attributi dal metodo:

Object.keys(obj)

oppure:

Object.getOwnPropertyName(obj)

6.21 Come modificare metodi e aggiungerne tramite prototype?

Per modificare un metodo (sovrascrivere) prima si salva il metodo precedente per evitare di ridefinirlo tutto e poi si fa così:

```
const vecchioSet = Map.prototype.set;
Map.prototype.set = function(key, value) {
  console.log(`Inserisco ${key} = ${value}`);

  return vecchioSet.call(this, key, value);
};
```

Invece, per aggiungere qualcosa basta fare così:

```
Map.prototype.print = function() {
  for (const [key, value] of this) {
    console.log(key, value);
  }
};
```

6.22 Qual è la differenza tra funzione normale e arrow function?

Entrambe rappresentano funzioni, ma hanno differenze importanti soprattutto riguardo al comportamento di **this**.

Una funzione normale possiede un proprio **this**, il cui valore dipende da come viene chiamata la funzione.

```
const persona = {
  nome: "Mario",
  saluta: function() {
    console.log(this.nome);
  }
};
```

```
persona.saluta(); // Mario
```

Una arrow function invece non crea un proprio **this**, ma eredita quello dello scope esterno (lexical this).

```
const saluta = () => {  
  console.log(this);  
};
```

Differenze principali:

- Le funzioni normali hanno un proprio **this**, le arrow function no.
- Le funzioni normali possono essere usate come costruttori con **new**, le arrow function no.
- Le funzioni normali possiedono un oggetto **prototype**, le arrow function no.

6.23 Come funziona il **this** in JavaScript?

this rappresenta l'oggetto che sta eseguendo una funzione. Il suo valore dipende dal contesto di chiamata.

Esempio come metodo di un oggetto:

```
const persona = {  
  nome: "Luca",  
  parla() {  
    console.log(this.nome);  
  }  
};  
  
persona.parla();
```

In questo caso **this** indica l'oggetto **persona**.

Nelle classi, **this** indica l'istanza corrente:

```
class Persona {  
  constructor(nome) {  
    this.nome = nome;  
  }  
}
```

Il valore di **this** può essere modificato tramite:

- `call()` esegue una funzione impostando manualmente `this`;
- `apply()` funziona come `call()` ma riceve gli argomenti come array;
- `bind()` crea una nuova funzione con `this` fissato.

6.24 Qual è la differenza tra shallow copy e deep copy?

La shallow copy è una copia superficiale di un oggetto: vengono copiati solo i valori del primo livello, mentre gli oggetti interni rimangono riferimenti condivisi.

Esempio:

```
const a = {  
  nome: "Mario",  
  indirizzo: {  
    citta: "Roma"  
  }  
};
```

```
const b = {...a};
```

Modificando `b.indirizzo.citta` viene modificato anche `a.indirizzo.citta`.

La deep copy invece crea una copia completa ricorsiva di tutti gli oggetti interni.

Esempio:

```
const b = structuredClone(a);
```

In questo caso gli oggetti interni non sono condivisi.

6.25 Qual è la differenza tra `==` e `===`?

L'operatore `==` effettua un confronto debole e permette la conversione automatica dei tipi.

Esempio:

```
5 == "5" // true
```

L'operatore `===` effettua un confronto stretto e confronta sia valore che tipo.

```
5 === "5" // false
```

Generalmente si preferisce utilizzare `===` perché evita conversioni automatiche.

6.26 Cos'è il type coercion in JavaScript?

Il type coercion è la conversione automatica dei tipi effettuata da JavaScript durante alcune operazioni.

Esempio:

```
"5" + 2 // "52"  
"5" - 2 // 3
```

Nel primo caso il numero viene convertito in stringa perché l'operatore `+` concatena stringhe.

Nel secondo caso la stringa viene convertita in numero.

6.27 Qual è la differenza tra null e undefined?

Entrambi rappresentano un'assenza di valore, ma hanno significati diversi.

`undefined` indica che una variabile esiste ma non ha ancora ricevuto un valore.

```
let x;  
console.log(x); // undefined
```

`null` indica invece un valore vuoto assegnato intenzionalmente.

```
let persona = null;
```

6.28 Cosa sono i tipi union e intersection in TypeScript?

Un union type permette a una variabile di assumere più tipi diversi.

```
let valore: string | number;
```

La variabile può contenere sia una stringa sia un numero.

Un intersection type invece combina più tipi e richiede tutte le proprietà dei tipi uniti.

```
interface Persona {  
    nome: string;  
}
```

```
interface Studente {  
    matricola: number;  
}
```

```
type PersonaStudente = Persona & Studente;
```

Il nuovo tipo deve contenere sia `nome` sia `matricola`.

6.29 Cos'è il type inference in TypeScript?

Il type inference è la capacità di TypeScript di dedurre automaticamente il tipo di una variabile senza dichiararlo esplicitamente.

Esempio:

```
let numero = 10;
```

TypeScript deduce automaticamente:

```
number
```

equivalente a:

```
let numero: number = 10;
```

6.30 Qual è la differenza tra type e interface in TypeScript?

Entrambi permettono di definire nuovi tipi.

Una interface è principalmente utilizzata per descrivere la struttura di oggetti.

```
interface Persona {  
    nome: string;  
}
```

Un type alias è più generale e permette anche di creare union e altri tipi complessi.

```
type Risultato = string | number;
```

Le interface possono essere estese e possono essere dichiarate più volte per essere unite automaticamente.

6.31 Cos'è una classe astratta (abstract class)?

Una classe astratta è una classe che non può essere istanziata direttamente.

Serve come base per altre classi.

```
abstract class Animale {  
    abstract verso(): void;  
}
```

Le classi figlie devono implementare i metodi astratti.

6.32 Cosa succede quando TypeScript viene compilato in JavaScript?

TypeScript viene trasformato in JavaScript tramite compilazione.

Durante questo processo vengono eliminati tutti gli elementi che esistono solo a compile-time:

- tipi;

- interface;
- generics;
- type alias.

Questo processo viene chiamato **type erasure**.
Le interface, ad esempio, non esistono a runtime.

6.33 Cos'è il runtime e cos'è il compile-time?

Il compile-time è il momento in cui il compilatore analizza il codice.
TypeScript controlla:

- correttezza dei tipi;
- errori di assegnazione;
- compatibilità tra interfacce.

Il runtime è invece il momento in cui il codice JavaScript viene eseguito.

6.34 Differenza tra Map e Object

Entrambi rappresentano strutture chiave-valore.

Gli oggetti utilizzano principalmente stringhe e simboli come chiavi.

Una Map permette invece di usare qualsiasi tipo come chiave.

```
const m = new Map();
```

```
m.set(1, "Mario");  
m.set({}, "valore");
```

Le Map inoltre:

- mantengono l'ordine di inserimento;
- possiedono metodi come `set()`, `get()` e `has()`;
- possiedono la proprietà `size`.

6.35 Cos'è una Promise e come funziona l'asincronia in JavaScript?

Una Promise rappresenta un valore che sarà disponibile nel futuro.

Possiede tre stati:

- pending;
- fulfilled;
- rejected.

Esempio:

```
const p = new Promise((resolve, reject) => {  
    resolve("ok");  
});
```

Le Promise vengono gestite tramite:

```
then()  
catch()
```

oppure tramite la sintassi:

```
async/await
```

6.36 Cos'è l'event loop?

JavaScript è un linguaggio single-threaded, quindi possiede un solo call stack.

L'event loop permette di gestire operazioni asincrone controllando quando il call stack è libero e inserendo le callback pronte per essere eseguite.

Il meccanismo comprende:

- Call Stack;
- Web API;
- Callback Queue;
- Event Loop.

6.37 Differenza tra stack, heap e garbage collector

Lo stack è una zona di memoria utilizzata per:

- chiamate di funzione;
- variabili primitive.

L'heap è utilizzato per memorizzare:

- oggetti;
- array;
- strutture dati complesse.

Il garbage collector libera automaticamente la memoria occupata da oggetti non più raggiungibili.

6.38 Cosa sono i decorator in TypeScript?

I decorator sono funzioni speciali che permettono di modificare o aggiungere comportamento a classi, metodi o proprietà.

Esempio:

```
@Log  
class Persona {}
```

Sono utilizzati soprattutto in framework come Angular.